Real-World Engineering Challenges #5

Resilient payments systems, large-scale data storage with OLTP and OLAP systems, platform team challenges, and more.

GERGELY OROSZ
AUG 30, 2022

👋 *Hi, this is Gergely with this month's free issue of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at big tech and high-growth startups through the lens of engineering managers and senior engineers. Subscribe to get weekly issues.* 👇

***'Real-world engineering challenges'** is a series in which I interpret interesting software engineering or engineering management case studies from tech companies. You might learn something new in these articles, as we dive into the concepts they contain.*

This series has been on hold for a few months. I'm experimenting with bringing it back, after feedback from people who say it helped them discover new engineering concepts.

Read earlier issues here: Real-world engineering challenges #1, #2, #3 and #4.

In this issue, we cover:

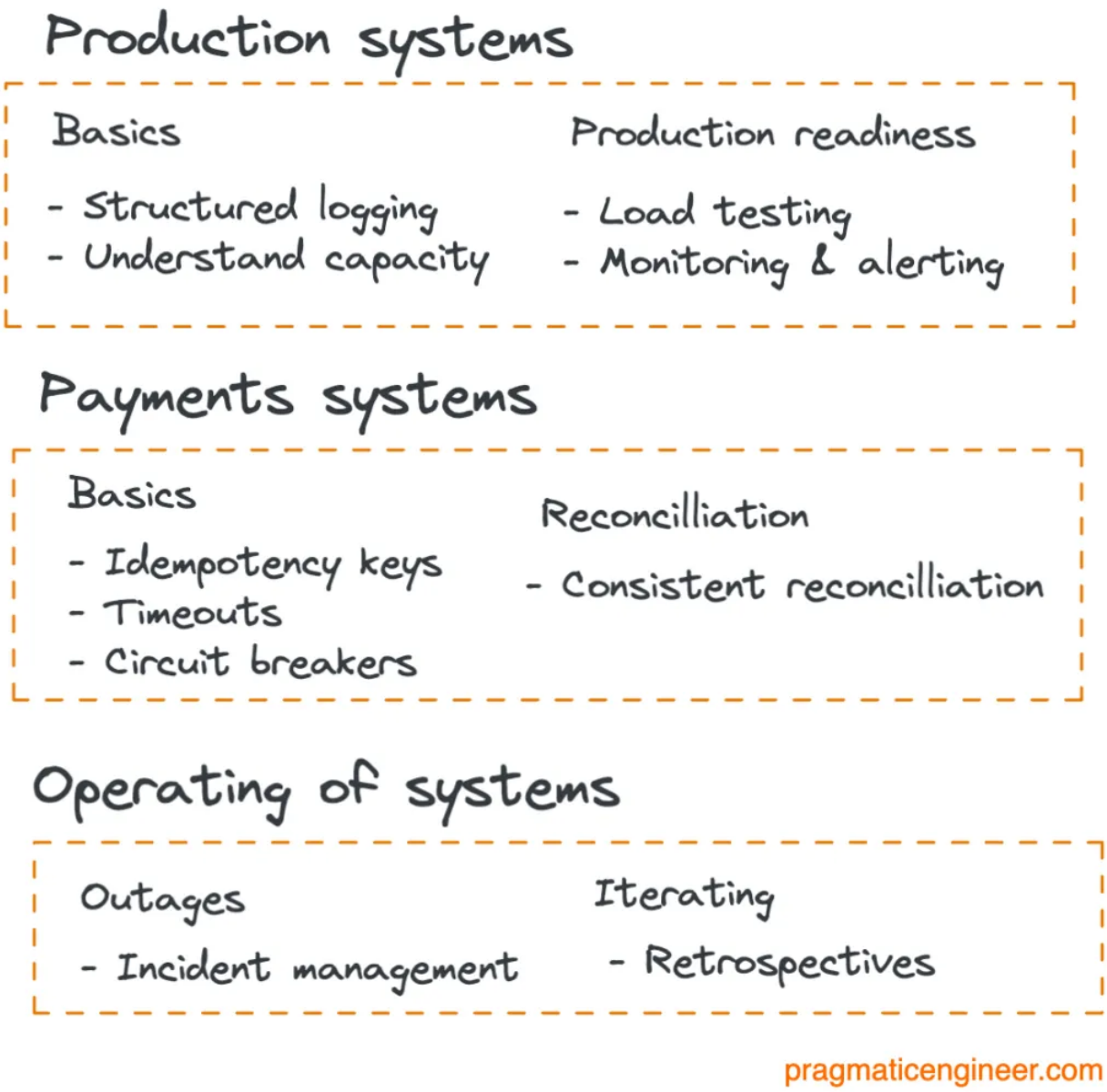
1. **Resilient payments systems learnings from Shopify.** Timeouts, circuit breakers, idempotency, reconciliation and more.
2. **Designing a solution to store and access millions of records by Grab.** Query and traffic patterns, OLTP (online transactional processing) and OLAP (online analytical processing) databases, and choosing the right technology for the right constraints.
3. **The challenges of the analytics infrastructure platform team at Yelp.** The trouble with validating platform work, not depending on client code and the

concept of onpoint week.

4. **Understanding how Instagram suggests content.** Content candidate generation using embedding and co-occurrence. Candidate selection using offline replays and online Bayesian optimizations, and fine-tuning the architecture.
5. **Centralizing incident management using Slack at Airbnb.** The company built a bot to centralize managing incidents across its over 1,000 microservices.
6. **An engineering manager's bill of rights at Honeycomb.** The engineering manager (EM) path offers less support in many organizations, than the senior individual contributor (IC) one. A suggestion on how to balance this out.

1. Resilient payments systems learnings from Shopify

Bart de Water worked as a staff software engineer on Shopify's payment infrastructure for more than five years. He boils down his observations to ten key learnings. I took the liberty of categorizing and visualizing them:



I'd argue many of the learnings are not limited to payments systems, but apply to any production system which fulfills important functions. Having healthy logging practices, and doing capacity planning ahead of time, all count as this. Before declaring a system is production-ready, ensuring monitoring and alerting are in place, and doing a load test – for large systems, at least – are prudent actions to take.

Capacity planning is a topic that's important for large systems. It's crucial to understand expected load in terms of QPS (queries-per-second,) and resource utilization in terms of CPU and memory and read and write loads, when choosing technologies and before going live with them. There are companies at which infra

teams provide resources to automatically scale workloads, and there are others where teams must allocate virtual machines ahead of time.

The author of the article makes the case that queue size, throughput and latency are all connected; so changing any one of them means a change elsewhere. **Rate limiting** and **load shedding** are approaches worth taking into account when dealing with additional traffic.

Some of the learnings are especially relevant for payments systems. **Idempotency keys** are needed to implement idempotent APIs – ones where double charges don't happen, even when requests are retried. Latencies and timeouts are especially relevant for real time payments systems. When systems are down, it can be better to not retry at all, and to use a **circuit breaker** to avoid cascading failures. Shopify built a Ruby circuit breaker called Semian, to protect its HTTP, MySQL and gRPC services.

The article touches on practices which are helpful when operating any production systems, such as having good incident management practices, or running regular incident retrospectives. *Previously in the newsletter, we covered Healthy oncall practices and Incident review and postmortem best practices.*

Payments systems are close to my heart, having worked on them at Uber for four years. I found myself nodding along with many of the learnings Bart has summarized.

The importance of idempotency for most payment systems is one I emphasize. A topic not covered in the list which I had many challenges with, is deciding when to fail open or to fail close with a payments system, and how to interpret unknown payments statuses. While at Uber, incorrect mapping of a payment provider's message allowed for free UberEats orders for two days in 2019, as I summarize in this video.

2. Designing a solution to store and access millions of records by Grab

How did Grab design its distributed system which processes incoming food orders, at a scale of millions of orders per day? Engineering manager Xi Chen and backend engineer Silang Cao wrote an in-depth article about this process. First, the team

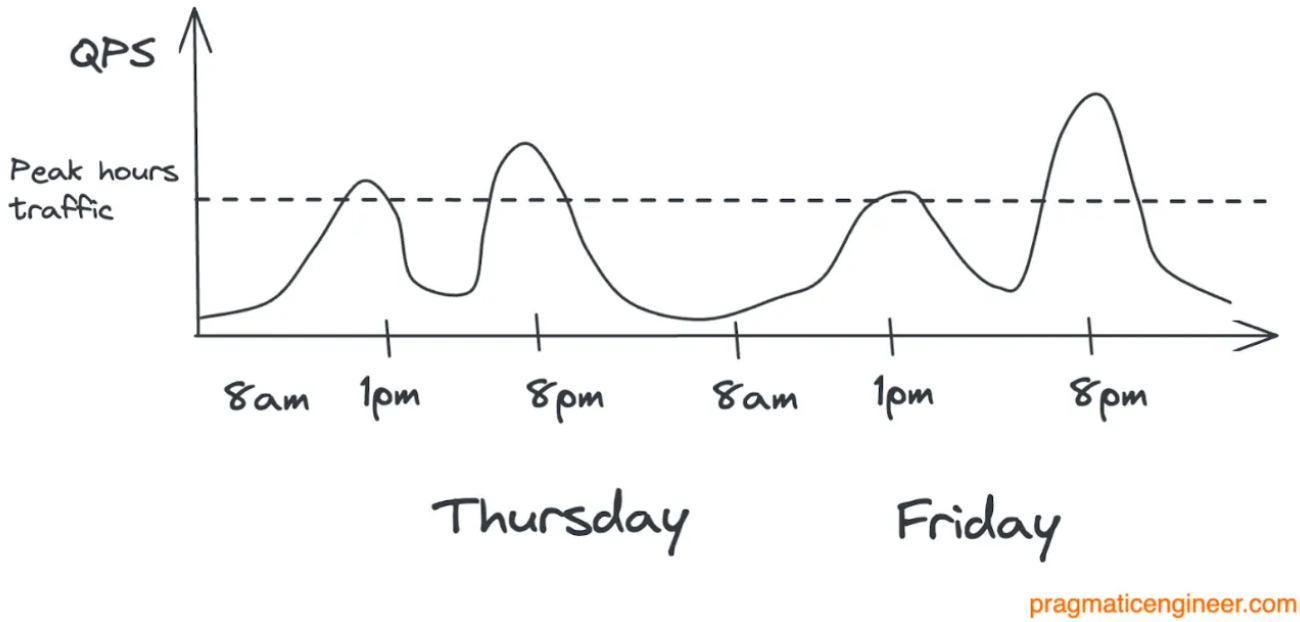
started by taking a step back, looking at the problem space and mapping out query and traffic patterns.

Query patterns. The team at Grab analyzed the type of **read queries** and **write queries** the platform supports. Queries could be split into **transactional queries** and **analytical queries**.

The differences between read/write and transactional/analytical queries are important because the ratio of read/write queries influences which data-level technology to use. Some technologies have better performance characteristics for read-intensive use cases, while others thrive with write-intensive use cases.

The transactional/analytical query split is important because transactional queries need to be as performant as possible. For analytical queries, performance may be a less important concern, so those queries don't need to be executed in real time.

Traffic patterns. Food delivery applications have traffic patterns where peak hours often result in several times the load being placed on systems. This is most easily detected by looking at the QPS load. For example, this is my sketch of how a food delivery service's order endpoints may receive data, with two distinct peaks at lunchtime and dinner time:



An illustration of a potential traffic pattern of food order endpoints, in a food ordering application. Peak hour load will be significantly higher, and these loads are driven by consumer behavior.

The Grab team found that during peak hours, write queries have triple the QPS compared to reads, suggesting a data layer approach is needed, which works well in write-intensive scenarios.

Design goals. Before choosing a solution, the team summarized the architectural priorities the solution needed to fulfill. They settled on stability even in high QPS situations, cost effectiveness and strong consistency for transactional queries.

Strong consistency means that in a distributed system, all nodes need to contain the same data at all times. Assuming a large enough system with several nodes, in practice this means trading off higher latency for strong consistency. This is because it takes more time for data changes to propagate to all nodes. Note that Grab didn't aim to build a system in which all queries have strong consistency. They only placed this constraint for transactional ones – but not for the analytical ones.

Technology choices. Grab chose an **OLTP database** which is optimized for high write load. The company chose DynamoDB because of characteristics like high availability, scalability, support for strong consistent reads by primary key and adaptive capacity to handle **hotkey traffic**.

Hotkey traffic refers to concentrated traffic directed at a hotspot of entries, instead of this traffic being evenly distributed. For databases that use sharding, this could mean overloading the nodes where these hotspots live.

An OLAP database was chosen to implement analytical queries. OLAP databases are optimized for high read loads. Grab used MySQL RDS for this approach.

The article has more detail on implementation, DynamoDB implementation (using GSI, sparse indexes, DynamoDB TTL), the reasons why Grab did not choose Aurora, how the company implemented data ingestion using Kafka streams, and the challenges of the current setup:

What I especially like about this article is how it showcases a data-first and principles-first design process. This case study showcases how it can be beneficial to split up your use cases and choose different technologies to support different use cases.

3. The challenges of building the analytics infrastructure team at Yelp

What is it like to work on an infrastructure platform team? Software engineers Alexander Dadukin and Dhriti Chawla at Yelp, wrote about their experience.

The two engineers make up the Android team of the Analytics Infra team. They own the Experimentation and Logging SDKs (software development kit), alongside other core modules. They share several of their challenges in the article.

They own no UI. All of their work is invisible, which is specifically challenging for mobile engineers, who are used to building visible things.

They cannot immediately validate their work. Their platform team depends on customer teams to use their features, and so can only validate that their improvements work when those customers adopt the features.

Of course, their team still writes tests, but validating work via tests is not the same as doing it in production. They use a sample app to test and roll out changes gradually. What's interesting is they share how rolling out their own platform features usually takes much longer than the timeframe in which product teams commonly roll out changes.

Platform code cannot depend on product code. The Yelp team phrases this as their SDK, a core dependency which cannot depend on client code inside other repositories.

The experimentation & logging SDK is the first thing to launch in the app. All parts of the app depend on experimentation and logging, so this part of the app is initialized first. The problem is that if this SDK crashes, the app is unrecoverable.

This means the platform team needs to ensure there is a very low – ideally zero – crash rate. They also need to find ways to report their own crashes in a way that does not

utilize the logging component they offer other teams for logging errors and crashes.

Onpoint week refers to a rotation system during which an engineer is responsible for answering questions on the team channel, I've confirmed with another engineer at Yelp. The team shares that this role is tiring and there are lots of questions.

What I find interesting about this article is that it gives a peek into the reality of working on a platform team. It showcases that platform teams are typically small – only two Android engineers on this team! – and that a lot of the work comes via communication with other teams.

The team reflected on how many questions they get from other teams, writing that while they have documentation, their system is complex and requires a little bit of understanding. My take is that the documentation might not be as intuitive as the team hoped for, or the API isn't as easy to use as they thought.

I was surprised to not read anything about this platform team gathering feedback from customers, then using that feedback to improve their SDK, documentation or processes. Doing this might result in a lower workload for the onpoint engineer.

It was curious that the article stresses the SDK cannot have other dependencies. I've observed that the fact platform code is considered "core," also gives an excuse to build things from scratch and not have other dependencies. Especially within Big Tech, you're far more likely to see all platform code implement things like networking, data storage and other things from scratch, over using third-party, open-source libraries. Also, in many cases, platform teams become intellectual playgrounds where reinventing the wheel is not scrutinized, as not doing so means there are no external dependencies.

The "use open-source" vs "build your own" debate won't ever go away. My view is you should decide which path to choose based on resources and constraints. I'd urge building things which are core to your business, and don't reinvent the wheel when there's no practical reason for it.

4. Understanding how Instagram suggests content

Instagram launched Suggested Posts in August 2020. Four months later in December 2020, the company shared a post about how it built this feature, which is essentially an **information retrieval system (IR)**.

Building information retrieval systems has two steps, both of which Instagram followed.

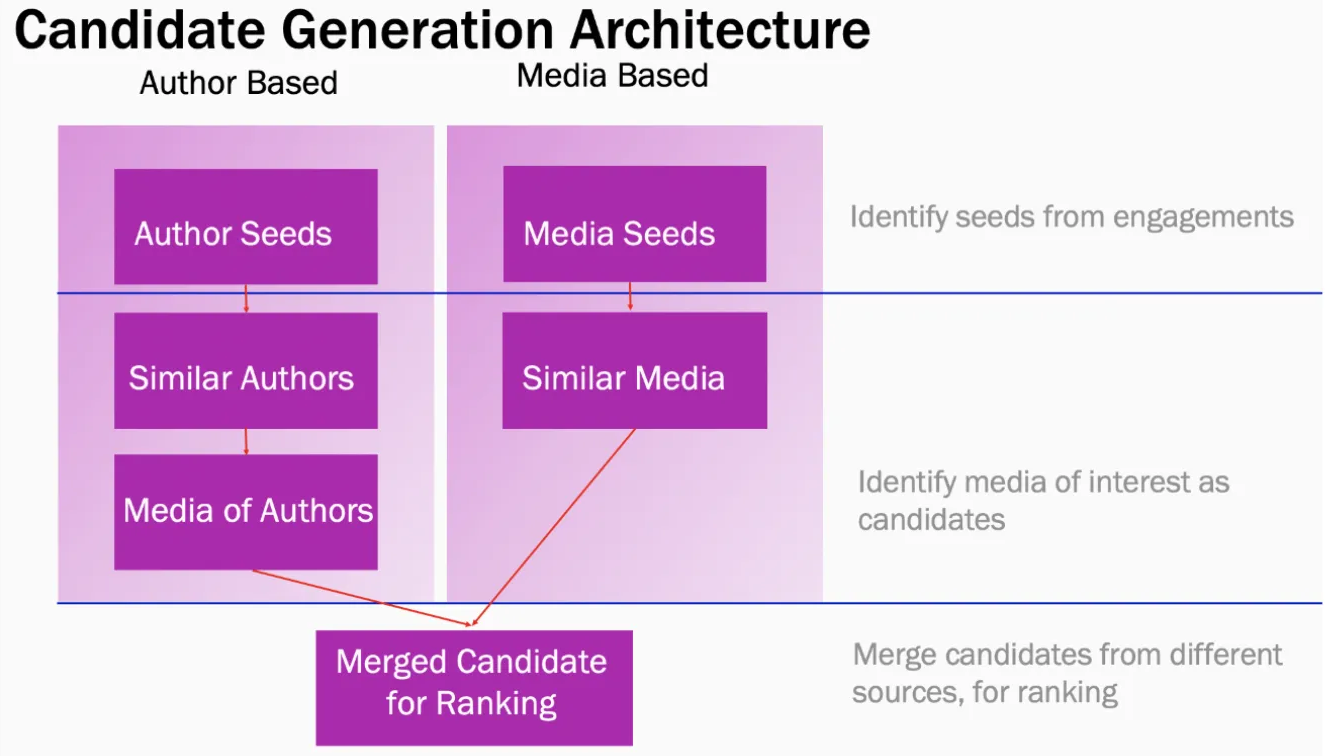
1. Candidate generation. Based on a user's implicit or explicit interests, grab all the candidates a user could possibly be interested in. This stage is heavy on **recall**.

Instagram builds a virtual graph of users' interests based on their engagement. The nodes in this graph are engagement actions such as a "liked" picture, a video saved, a friend's post liked or friend's post saved.

Seeds in this graph are the nodes the user has shown interest in, the nodes of the engagement actions. Seeds then can be used as input for the **K-nearest-neighbor** (KNN) pipelines. Instagram builds these KNN pipelines based on two classic machine learning principles:

- **Embeddings** based on similarity. An embedding is a fundamental structure of recommendation systems; it's a relatively low-dimensional space into which higher dimensional vectors can be translated. Instagram builds embeddings based on user engagement data, to help find thematically and topically relevant accounts.
- **Co-occurrence**-based similarity. A co-occurrence matrix computes the number of times each pair of items appear together in a list. Instagram generates a matrix using user-media interaction data, then calculates co-occurrence frequencies on media pairs.

This is what the Suggested Posts candidate generation architecture looks like:



Candidate generation architecture for Suggested Posts. Source: Facebook's engineering blog

2. Candidate selection. From all the candidates generated, select the best subset for the user. This stage is typically done by a heavyweight ranking algorithm.

Instagram does the weighting based on both positive engagement factors for each node - such as "like", comment, save. It does this for negative factors as well, like 'not interested' and 'show fewer posts like this.'

Now you know what happens when you hit the 'not interested' button: you're adding negative weight to a given node in the candidate selection process!

The weights are tuned in two ways:

- **Offline replay** over offline user sessions. Offline replay most likely refers to experience replay in Q-learning algorithms.
- **Bayesian optimization** during online optimization. Bayesian optimization is a frequently used approach in applied machine learning. It is based on the Bayes theorem. Here's a video explaining how Bayesian optimization works.

For model classes, Instagram uses:

- **Multi-task multi-label sparse neural nets** (MTML). Here's an overview of multi-task learning (MTL) and a paper on MTML.
- **Gradient-boosted decision trees** (GBDT.) Gradient boosting builds simpler prediction models sequentially, where each model attempts to predict the error of the previous model. Read an overview of an introduction to GBDT.

Instagram tunes the architecture and its hyperparameters during training, offline replay and online A/B tests. They also experiment with other approaches like multi-stage ranking and distillation models – a form of model compression.

Although the article feels "hand-wavy" with few specific implementation details, I find it a useful collection of the wide variety of machine learning concepts and approaches which Instagram uses in large-scale recommendation systems. It's also a reminder of how broad and deep applied machine learning is. If you're a software engineer or engineering manager working with, or wanting to work with, machine learning engineers, these are topics worth familiarizing yourself with.

5. Centralizing incident management using Slack at Airbnb

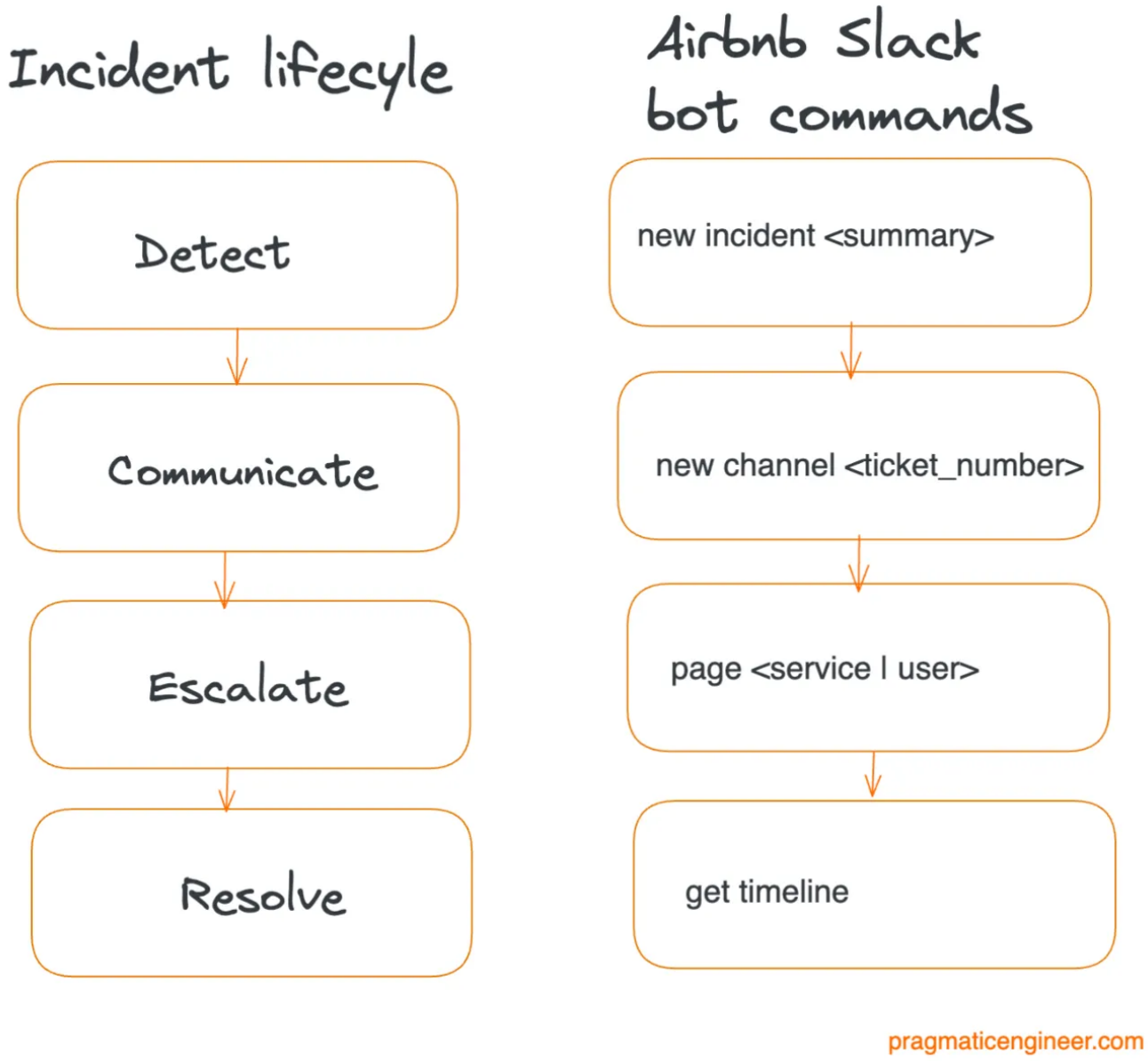
Airbnb runs more than 1,000 microservices underneath its hood. Different teams own different services, teams own multiple services, and ownership of these services can change, as teams re-organize. Site reliability engineer Vlad Vassiliouk summarized how and why Airbnb automated incident management using a custom Slack bot.

A problem that Airbnb found is that teams spent a lot of time switching between Slack, Pagerduty and JIRA in order to:

1. Declare incidents
2. Find and page the appropriate responders
3. Provide context for the relevant parties in the incident

The company developed an incident management bot, moving all incident management from three tools – Slack, PagerDuty, JIRA – to one: Slack.

The Slack bot has four main commands, which all map to typical incident management lifecycle states:



The bot has some handy automations, such as offering to page incident managers, and capturing incident details from within Slack. Check out the full article for more details on how the bot works and the handy shortcuts that Airbnb implemented.

I liked reading about how the SRE (site reliability engineering) team rolled up their sleeves and built a tool to simplify the lives of oncall engineers, because this is what great SRE teams do. They make the jobs of engineers easier, especially when it comes to alerting, monitoring, incidents, and all the work surrounding this.

Slack is a flexible enough product that it's easy enough to build custom integrations like Airbnb did. Of course, as with any generic enough tooling, there's the question of whether to build such integrations yourself, or buy an off-the-shelf solution. When it comes to incident management tooling using Slack, there is no shortage of startups that are either Slack-first, or offer Slack-native integrations. Just a few examples include incident.io (disclosure: I'm an investor), Jeli, Rootly, FireHydrant and Kintaba. Read responses on this Twitter thread for more alternatives.

In the case of Airbnb, things like implementing a *page <service | team>* is something that seems highly specific to the company. Behind the scenes, that command likely talks to one or more systems that look up service ownership and uses Pagerduty's API to locate oncall engineers to ping. This custom, behind-the scenes-part, could be more complex than the integration itself, which means that building this tool in-house could have been the sensible choice: for Airbnb, that is.

6. An Engineering Manager's Bill of Rights by Honeycomb's VP Eng

Honeycomb's VP of Engineering Emily Nakashima touches on a neglected subject: that the engineering manager (EM) path offers less support in many organizations than the senior individual contributor (IC) one does. As she writes:

> "Many of the best and most promising managers I know have left management roles for senior IC roles since 2018. (..) I also observe that a truly staggering number of Honeycomb's most effective, most admired senior ICs are former managers (...)
>
> It is also a sign that the job of engineering manager has gotten harder, and in many organizations, it can now feel like an unwinnable game, caught between two distinct and sometimes conflicting generations of expectations."

In this article, she first summarizes the additional expectations engineering managers are typically measured against: emotional intelligence, creating psychological safety, supporting employees in crises, responding with care and empathy. And – of course – coaching, mentoring and career development.

She then proposes a contract for an organization to offer engineering managers; a way to counterbalance the ever-growing expectations EMs work under. Her manifesto is below, **emphasis** mine:

1. **Culture of respect**. Build a culture of respect for both management and IC work.
2. **Multiple advancement paths**. Provide a career path within management that has multiple ways to advance.
3. **Clarity**. Be straight with teams about what managers are there to do.
4. **Feedback from ICs as well.** Cultivate the flow of feedback in all directions and remind ICs they should expect to give feedback to their managers, not just receive it.
5. **EM compensation to move with the market**. Allow manager & IC compensation to move independently with the market.
6. **Transparency with guaranteed response to managers**. Provide transparency and guarantee a response to our teams, even when the answer isn't easy.
7. **Clarify the EM's responsibilities**. Be clear on what these are.

Read the full article for more nuance on all the above points and to find out which is the most controversial clause on this list, according to the author.

This article really strikes a chord with me. When I became an engineering manager at Uber, I was enthusiastic about my new role, initially. However, over time I started to feel some parts of the job made it more difficult to progress, than if I would have stayed an IC. The three biggest issues I saw were:

- Feeling trapped as a middle manager. Leadership would sometimes give orders from above with no transparency on the "why," and didn't respond to questions. This left me in an uncomfortable position where I did not know why I was to do the things asked of me. #6 in this EM Bill of Rights addresses exactly this.

- Lack of feedback. When you become an EM, you get a lot less feedback. I like how #4 in Emily's list focuses on ensuring feedback keeps flowing from ICs, as well.
- Compensation tied to engineering. At Uber, compensation bands for EMs were the same as for ICs. While this was never my biggest concern, I would support making the two independent as suggested in #5, so they can move with the market, in-line with demand for senior ICs and managers.

Takeaways

I revived the Real World Engineering Challenges series after feedback from readers who shared how they enjoyed the articles as a way to keep their tech chops sharp, and to get exposure to engineering and organizational case studies outside of their everyday "work bubble."

When I started The Pragmatic Engineer newsletter almost a year ago, I committed to one long-form article every Tuesday for subscribers. I've since quietly changed this cadence to two longer articles on Tuesdays and Thursdays. The Tuesday ones are more educational, and are typically deep dives into a given topic. The Thursday articles tend to report the latest news – including exclusive stories from inside Big Tech and high-growth startups – and they also reflect on recent events. This is The Scoop series.

Going forward, I'm trialing adding a Real World Engineering Challenges article to Tuesday's publishing schedule. This series would run every 4-6 weeks. Tuesday is when the more educational and deep-dive articles come out. So, I'm interested in your feedback on how useful you found this newsletter. Please give feedback using the links below.

How would you rate this issue? 🤔

Amazing • Great • Good • OK • So-so

74 Likes

4 Comments

Write a comment...

Ayush Tickoo Aug 30, 2022

This post came out more like an aggregation of good articles. I really look forward to your exclusives. Personally, I did not enjoy it that much. Not sure, if this is the right place to share feedback.

LIKE (1) REPLY

2 replies by Gergely Orosz and others

Dmitry Verkhoturov Oct 12, 2022

The format is heavy to digest as the post is lengthy and heavy with information, and I did not click any links, but your summaries are interesting

LIKE REPLY

2 more comments...